Assignment 1: Critique

Devansh Shah (dns5508)

Collaboration and AI use statement

For understanding this assignment, I used Claude Code by Anthropic. I used it to walk me through the basic and sweep scenario in complete detail. Then, I used it to understand the flow and logic of the code. I also provided the design document and asked questions about it to strengthen my understanding. I also used Claude's help to write a python script to compare tick-to-trade latency using the current method and the `capture.csv` method, to observe the gap between the two evaluation approaches. My friend Sarvesh and I discussed our understanding and perspectives for better clarity.

Describe one possible execution of the hotpath.

Taking the basic.scn example, a competitor sends a bid of 100.01 for 100 shares. `OnMarketData` decodes the message and checks the sequence number. The order book and the best bid are updated. Next, `Requote` checks the book is valid, then calculates the fair value and credit. For each side, `QuoteSide` checks the position limit and whether the quote has changed. Since the bid price has not changed, no order is sent for the bid. However, the ask price changes from 100.04 to 100.03. `SendQuote` records the decision, writes to the decision log, and sends a replace request. The capture tap records the send time. The handler then records its completion time and returns.

Describe a flaw in the evaluation and a potential fix.

The evaluation measures the time taken by the handler to process a message. The design justifies averaging over every message because messages that do not trade still cause work. It also measures until the handler returns, and not till when the order is actually sent. However, tick-to-trade is the time from a message arriving to the order being sent, so messages that send no order should not be part of it, as they dilute the average.

A potential fix is the capture tap already records the delivery entry time, the order send time, and which message was being handled when the order was sent. I would use these timestamps to directly calculate the latency for each order that is actually sent. I would also measure the median, p99, and maximum latency instead of only the average, since the average hides rare slow cases. This would give an accurate picture of the jitter and latency of the system.

Describe a good design decision in the market maker.

One good design decision is storing orders in a flat array indexed by the exchange order reference instead of using a hash map. The array lets the system find an order directly without hashing or following pointers to separately allocated nodes. This works because exchange references are issued in increasing order, so the array is relatively dense. A lookup is a single indexed load, so it costs at most one cache miss. The 16 MB array is allocated and zeroed at startup, so there is no allocation overhead during message processing, but its capacity is fixed at about a million references.

Describe a flaw in the market maker and a potential fix.

The risk checks use exceptions to signal problems. The design justifies this by saying exceptions cost nothing when not thrown, and that once one is thrown we are no longer on the hotpath. However, this is not true. If the bid side is blocked by the position limit, the exception is caught and the ask side is still quoted right after, so the throw sits on the tick-to-trade path. An invalid book also throws on every message while a side is empty. Since the position check runs before the check for an unchanged quote, a position near the limit throws on every message, even when no order needs to be sent. Throwing and catching is an expensive operation.

Instead of throwing exceptions, I would update the checks to return a status. The caller can then branch on it, which is a predictable compare so minimal cost. Marking the return value as `nodiscard` would also help prevent callers from ignoring it.

Describe a second flaw in the market maker and a potential fix.

The market maker writes every order to `decisions.log` using `fprintf` before sending it. This adds formatting and I/O overhead to the tick-to-trade path. Also, since the log is buffered inside the process, recent entries can be lost if the process crashes. This means we pay the latency cost of logging without getting the full guarantee of a write-ahead log.

I would replace `fprintf` with a binary record copied with `memcpy` into a preallocated memory-mapped file. This removes the formatting cost and the occasional write when the buffer fills up. This method also survive a process crash.